Mars 블록체인 전체 애플리케이션 흐름도

개요

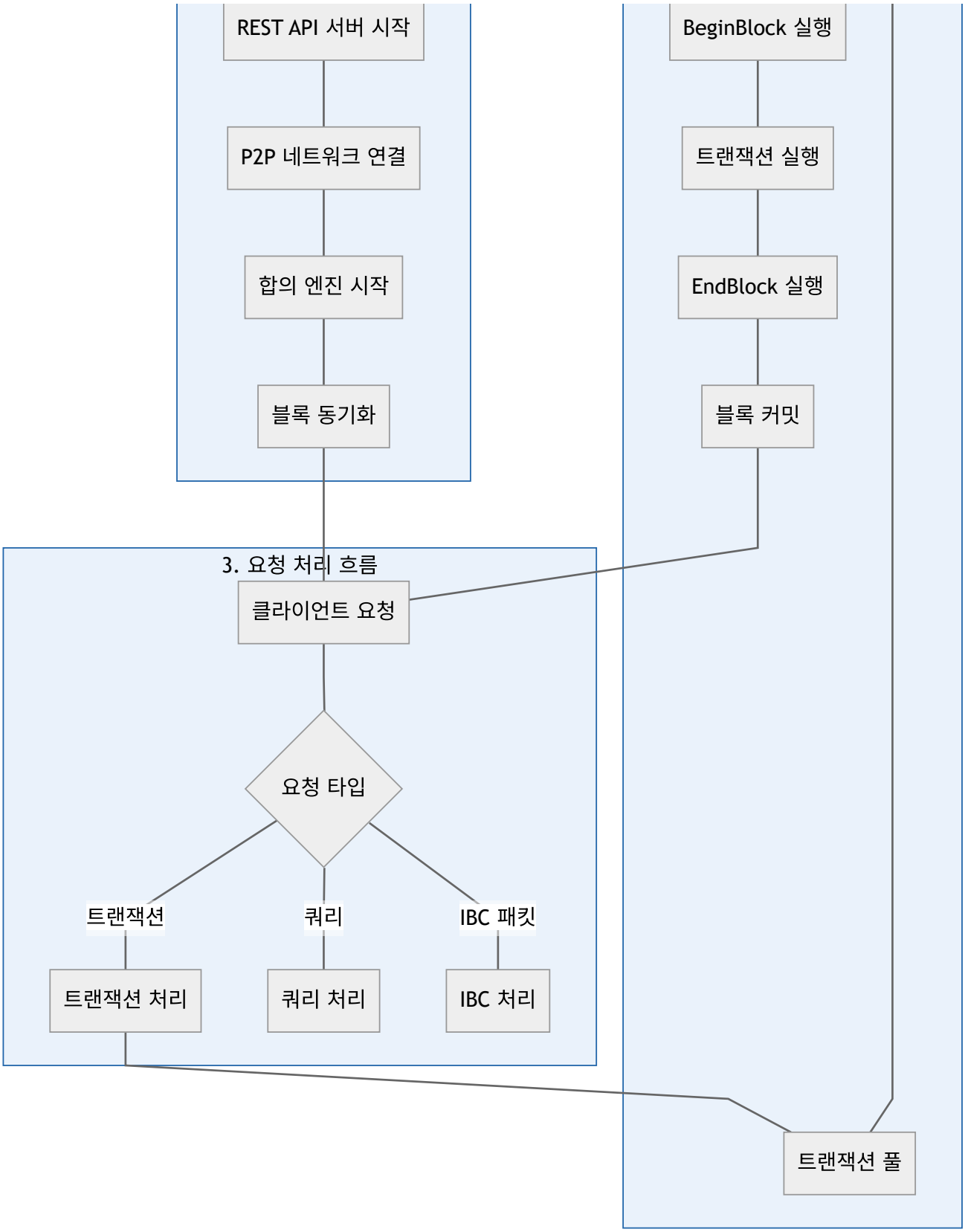
이 문서는 Mars 블록체인 애플리케이션의 전체 흐름을 시각화하고 설명합니다. 앱 초기화부터 요청 처리까지의 모든 단계를 포함하며, 각 단계별 상세 설명과 코드 참조를 제공합니다.

주요 특징:

- **의존성 주입 기반 아키텍처**: `depinject` 시스템을 통한 자동 의존성 해결
- **모듈화된 설계**: 각 기능별로 독립적인 모듈 구조
- **IBC 지원**: 크로스체인 상호 운용성을 위한 Inter-Blockchain Communication
- **Collections 기반 상태 관리**: 타입 안전한 상태 저장소 시스템
- **낙관적 실행**: 트랜잭션 병렬 처리를 통한 성능 최적화

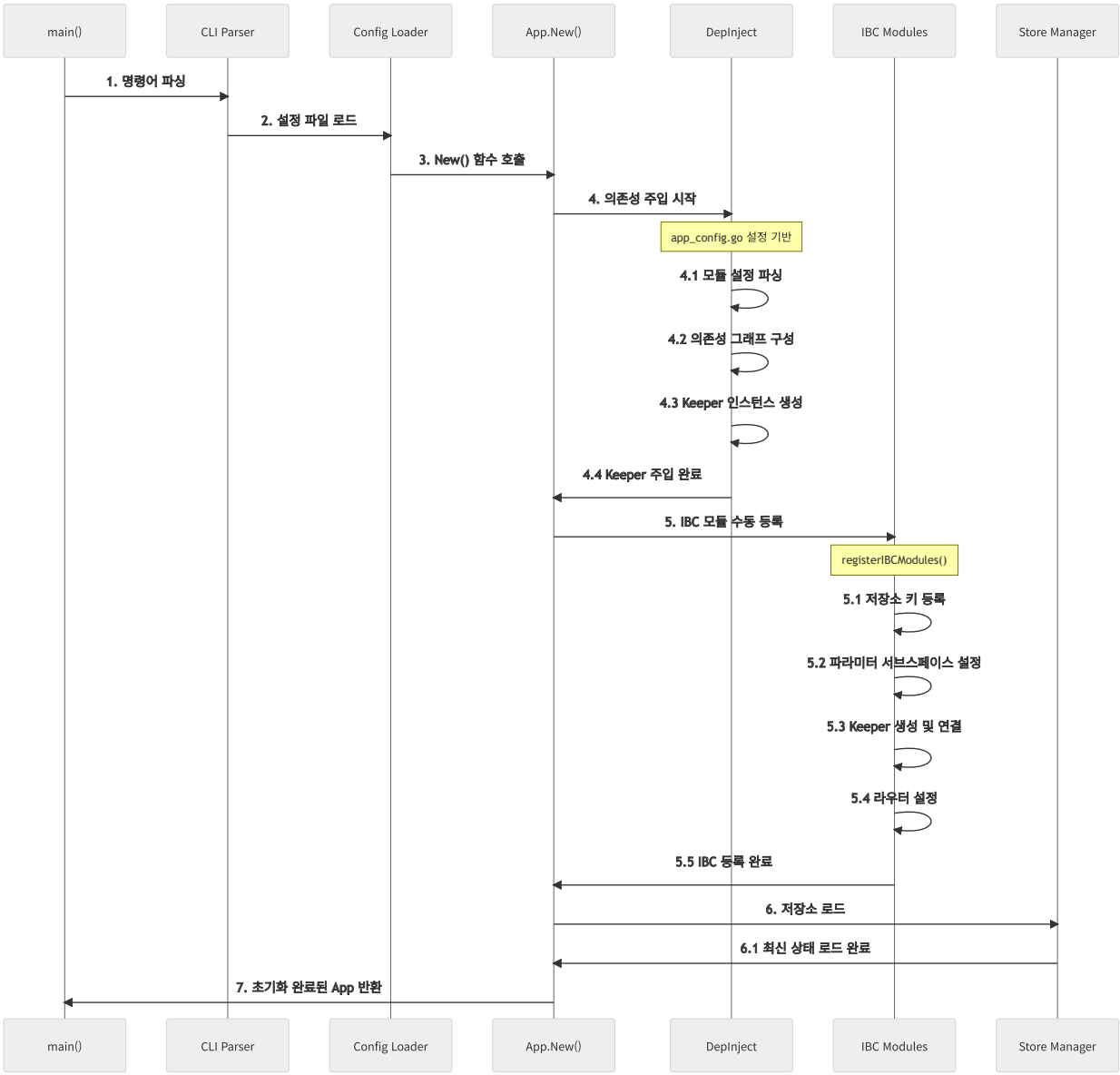
1. 전체 시스템 아키텍처 흐름도

1. 애플리케이션 초기화
main 함수 시작
CLI 명령어 파싱
설정 파일 로드
New 함수 호출
의존성 주입 실행
IBC 모듈 수동 등록
시뮬레이션 관리자 설정
앱 로드 완료
2. 서버 시작
BaseApp 초기화
gRPC 서버 시작
4. 블록 생성 및 합의
블록 제안
PreBlock 실행



2. 애플리케이션 초기화 상세 흐름

2.1 초기화 단계별 분석


main()
CLI Parser
Config Loader
App.New()
DepInject
IBC Modules
Store Manager
1. 명령어 파싱
2. 설정 파일 로드
3. New() 함수 호출
4. 의존성 주입 시작
app_config.go 설정 기반
4.1 모듈 설정 파싱
4.2 의존성 그래프 구성
4.3 Keeper 인스턴스 생성
4.4 Keeper 주입 완료
5. IBC 모듈 수동 등록
registerIBCModules()
5.1 저장소 키 등록
5.2 파라미터 서브스페이스 설정
5.3 Keeper 생성 및 연결
5.4 라우터 설정
5.5 IBC 등록 완료
6. 저장소 로드
6.1 최신 상태 로드 완료
7. 초기화 완료된 App 반환
main()
CLI Parser
Config Loader
App.New()
DepInject
IBC Modules
Store Manager

2.2 의존성 주입 상세 과정

코드 참조: `app/app.go:New()` 함수 (라인 200-250)

```
// 1단계: 기본 변수 초기화
var (
app = &App{}
appBuilder *runtime.AppBuilder
appConfig = depinject.Configs(
AppConfig(), // app_config.go에서 정의된 모듈 설정
depinject.Supply(appOpts, logger),
)
)

// 2단계: 의존성 주입 실행
if err := depinject.Inject(appConfig,
&appBuilder,
&appModules,
```

```
// 코덱 시스템
&app.appCodec, // Protocol Buffers 코덱
&app.legacyAmino, // 레거시 Amino 코덱
&app.txConfig, // 트랜잭션 설정
&app.interfaceRegistry, // 인터페이스 레지스트리
// 핵심 Keeper들 (의존성 순서대로 주입)
&app.AuthKeeper, // 계정 관리
&app.BankKeeper, // 토큰 전송
&app.StakingKeeper, // 스테이킹
&app.SlashingKeeper, // 슬래싱
&app.MintKeeper, // 토큰 발행
&app.DistrKeeper, // 보상 분배
&app.GovKeeper, // 거버넌스
&app.MarsKeeper, // Mars 커스텀 모듈
// ... 기타 keeper들
); err != nil {
panic(err)
}
```

의존성 해결 순서:

1. **모듈 설정 파싱**: `app_config.go` 의 `appConfig` 구조체 분석
2. **의존성 그래프 구성**: 모듈 간 의존성 관계 매핑
3. **Keeper 생성 순서**: 의존성이 없는 모듈부터 순차적 생성

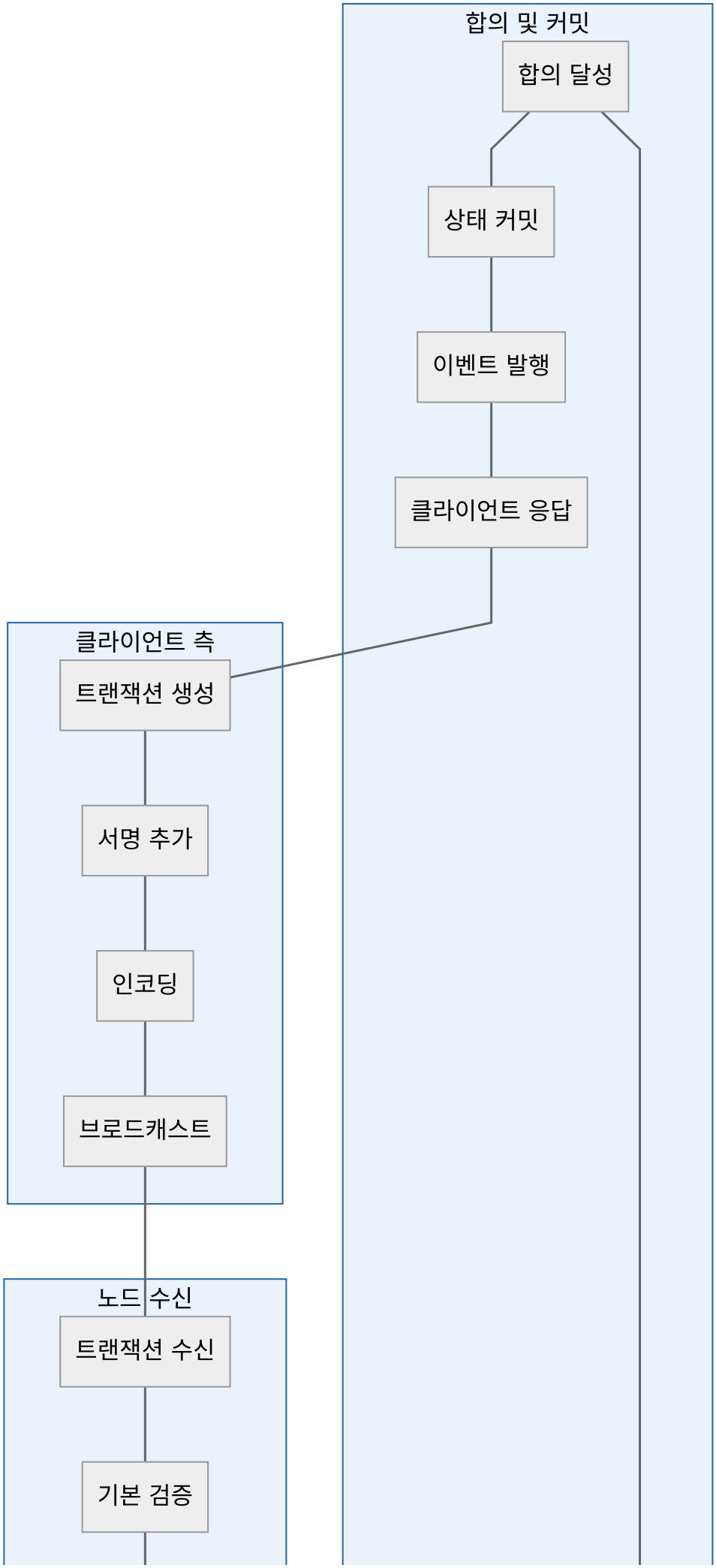
4. **의존성 주입**: 각 Keeper에 필요한 다른 Keeper들 주입

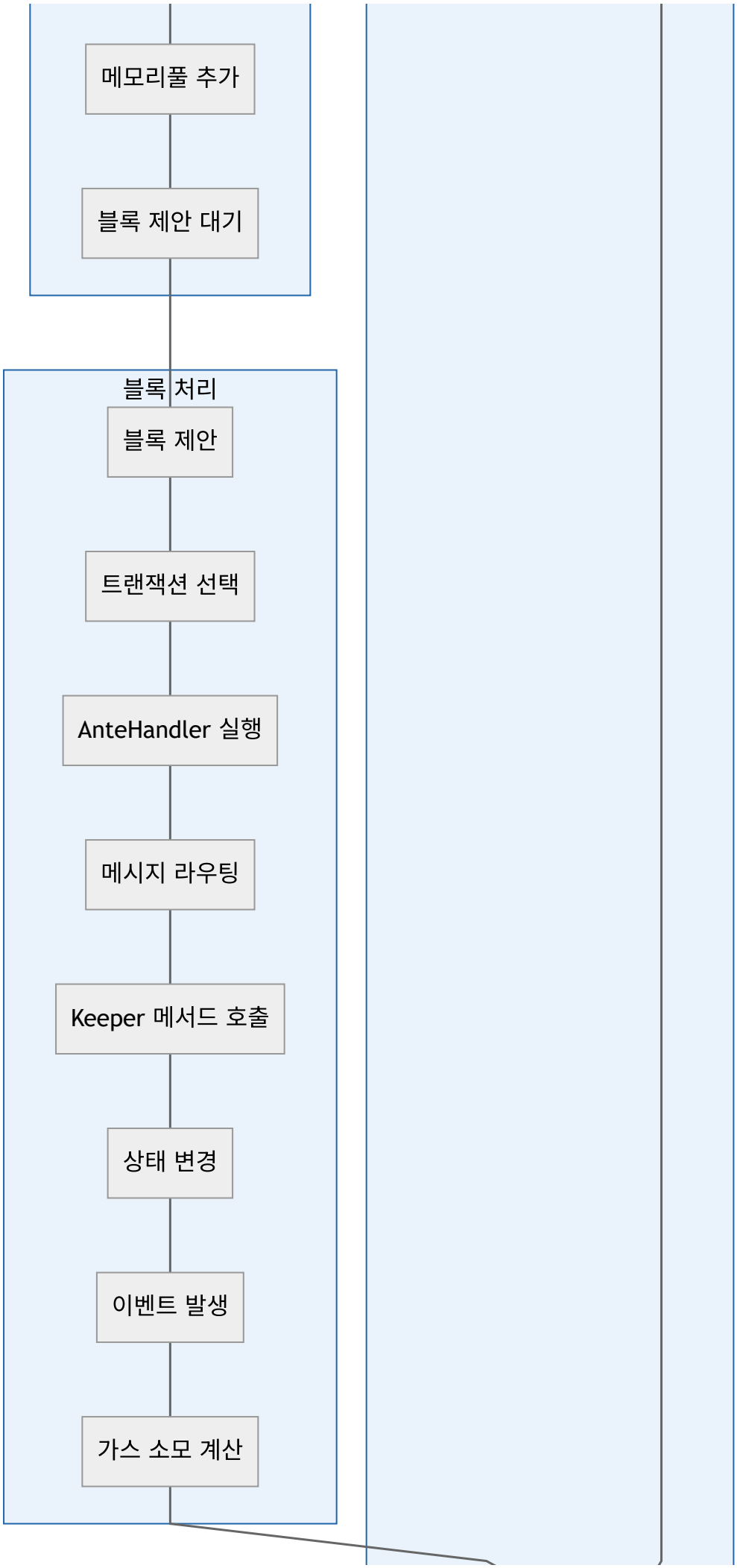
주요 의존성 관계:

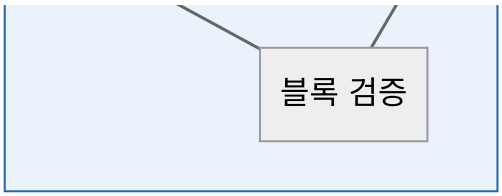
- `BankKeeper` → `AuthKeeper` (계정 정보 필요)
- `StakingKeeper` → `BankKeeper` (토큰 전송 필요)
- `SlashingKeeper` → `StakingKeeper` (검증자 정보 필요)
- `DistrKeeper` → `StakingKeeper`, `BankKeeper` (보상 분배 필요)
- `GovKeeper` → 모든 모듈 (거버넌스 제안 처리 필요)

3. 트랜잭션 처리 흐름

3.1 트랜잭션 라이프사이클

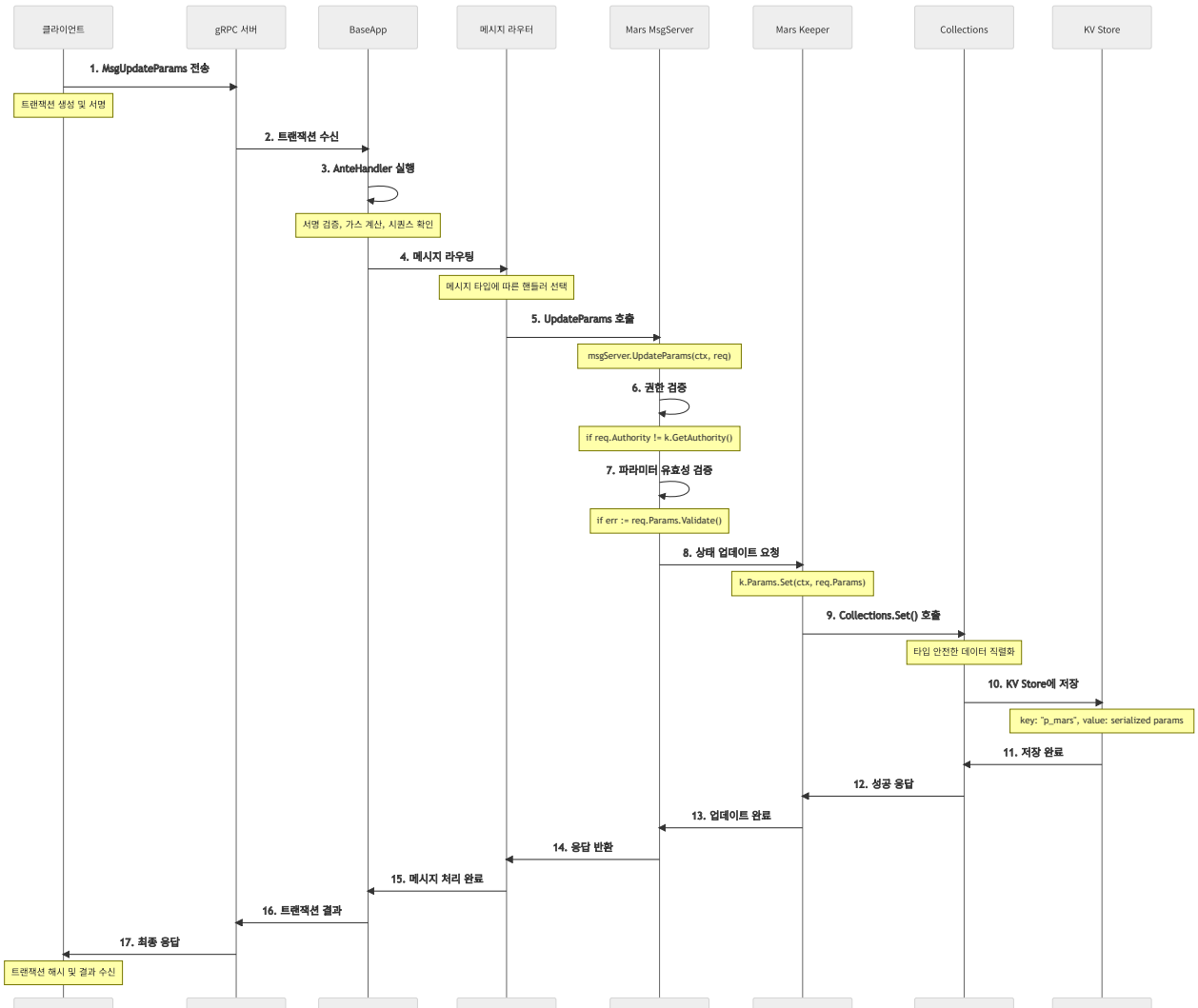

합의 및 커밋
합의 달성
상태 커밋
이벤트 발행
클라이언트 응답
클라이언트 측
트랜잭션 생성
서명 추가
인코딩
브로드캐스트
노드 수신
트랜잭션 수신
기본 검증


메모리풀 추가
블록 제안 대기
블록 처리
블록 제안
트랜잭션 선택
AnteHandler 실행
메시지 라우팅
Keeper 메서드 호출
상태 변경
이벤트 발생
가스 소모 계산



3.2 메시지 처리 상세 흐름

코드 참조: `x/mars/keeper/msg_server.go` , `x/mars/keeper/msg_update_params.go`


클라이언트
gRPC 서버
BaseApp
메시지 라우터
Mars MsgServer
Mars Keeper
Collections
KV Store
1. MsgUpdateParams 전송
트랜잭션 생성 및 서명
2. 트랜잭션 수신
3. AnteHandler 실행
서명 검증, 가스 계산, 시퀀스 확인
4. 메시지 라우팅
메시지 타입에 따른 핸들러 선택
5. UpdateParams 호출
msgServer.UpdateParams(ctx, req)
6. 권한 검증
if req.Authority != k.GetAuthority()
7. 파라미터 유효성 검증
if err := req.Params.Validate()
8. 상태 업데이트 요청
k.Params.Set(ctx, req.Params)
9. Collections.Set() 호출
타입 안전한 데이터 직렬화
10. KV Store에 저장
key: "p_mars", value: serialized params
11. 저장 완료
12. 성공 응답
13. 업데이트 완료
14. 응답 반환
15. 메시지 처리 완료
16. 트랜잭션 결과
17. 최종 응답
트랜잭션 해시 및 결과 수신

메시지 처리 핵심 단계:

1. **AnteHandler 검증** (`app/app.go`):

- 서명 유효성 검증
- 가스 한도 확인
- 계정 시퀀스 검증
- 수수료 차감

2. **권한 검증** (`x/mars/keeper/msg_update_params.go`):

```
if req.Authority != k.GetAuthority() {
return nil, govtypes.ErrInvalidSigner
}
```

3. **파라미터 검증**:

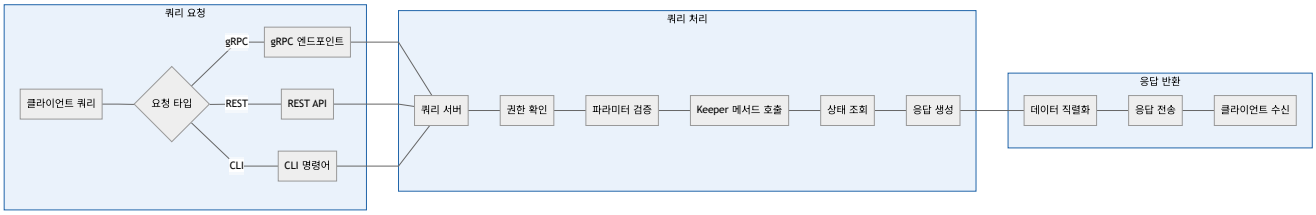
```
if err := req.Params.Validate(); err != nil {
return  nil, err
}
```

4. **상태 업데이트** (`x/mars/keeper/keeper.go`):

```
// Collections를 통한 타입 안전한 저장
if err := k.Params.Set(ctx, req.Params); err != nil {
return  nil, err
}
```

4. 쿼리 처리 흐름

4.1 쿼리 라이프사이클



4.2 Mars 모듈 쿼리 처리

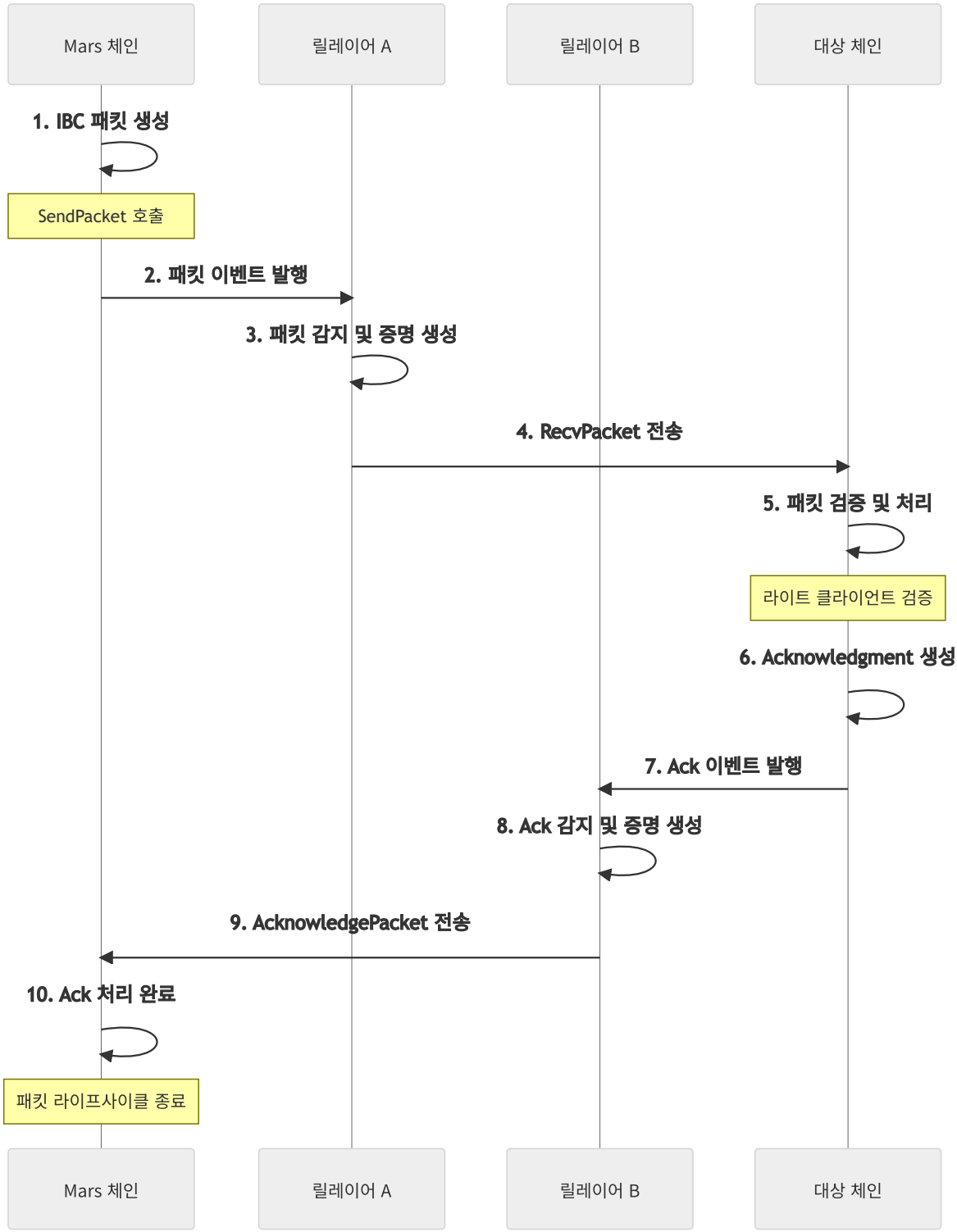
코드 참조: `x/mars/keeper/query.go`

```
// QueryParams는 모듈 파라미터를 조회합니다
func (k Keeper) QueryParams(ctx context.Context, req *types.QueryParamsRequest)
// 1. 요청 유효성 검증
if req == nil {
return nil, status.Error(codes.InvalidArgument, "invalid request")
}

// 2. 상태에서 파라미터 조회
params, err := k.Params.Get(ctx)
if err != nil {
return nil, status.Error(codes.Internal, err.Error())
}
```

```
// 3. 응답 생성 및 반환

return &types.QueryParamsResponse{Params: params}, nil

}
```

5. IBC 패킷 처리 흐름

5.1 크로스체인 통신 흐름


Mars 체인
릴레이어 A
릴레이어 B
대상 체인
1. IBC 패킷 생성
SendPacket 호출
2. 패킷 이벤트 발행
3. 패킷 감지 및 증명 생성
4. RecvPacket 전송
5. 패킷 검증 및 처리
라이트 클라이언트 검증
6. Acknowledgment 생성
7. Ack 이벤트 발행
8. Ack 감지 및 증명 생성
9. AcknowledgePacket 전송
10. Ack 처리 완료
패킷 라이프사이클 종료
Mars 체인
릴레이어 A
릴레이어 B
대상 체인

5.2 IBC Transfer 처리 과정

코드 참조: `app/ibc.go`

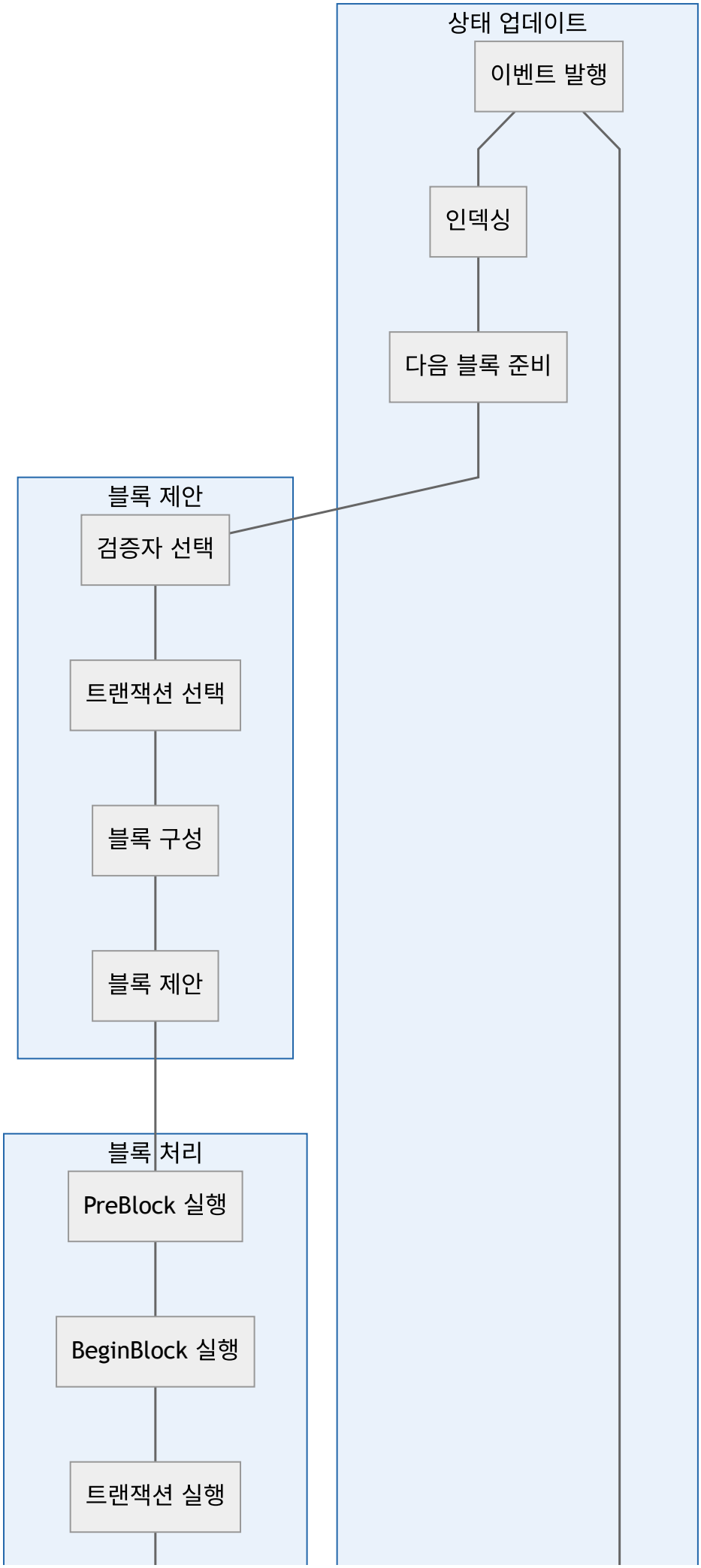
```
// IBC Transfer 모듈 스택 구성
transferStack := ibctransfer.NewIBCModule(app.TransferKeeper)

// 라우터에 등록
ibcRouter.AddRoute(ibctransfertypes.ModuleName, transferStack)
```

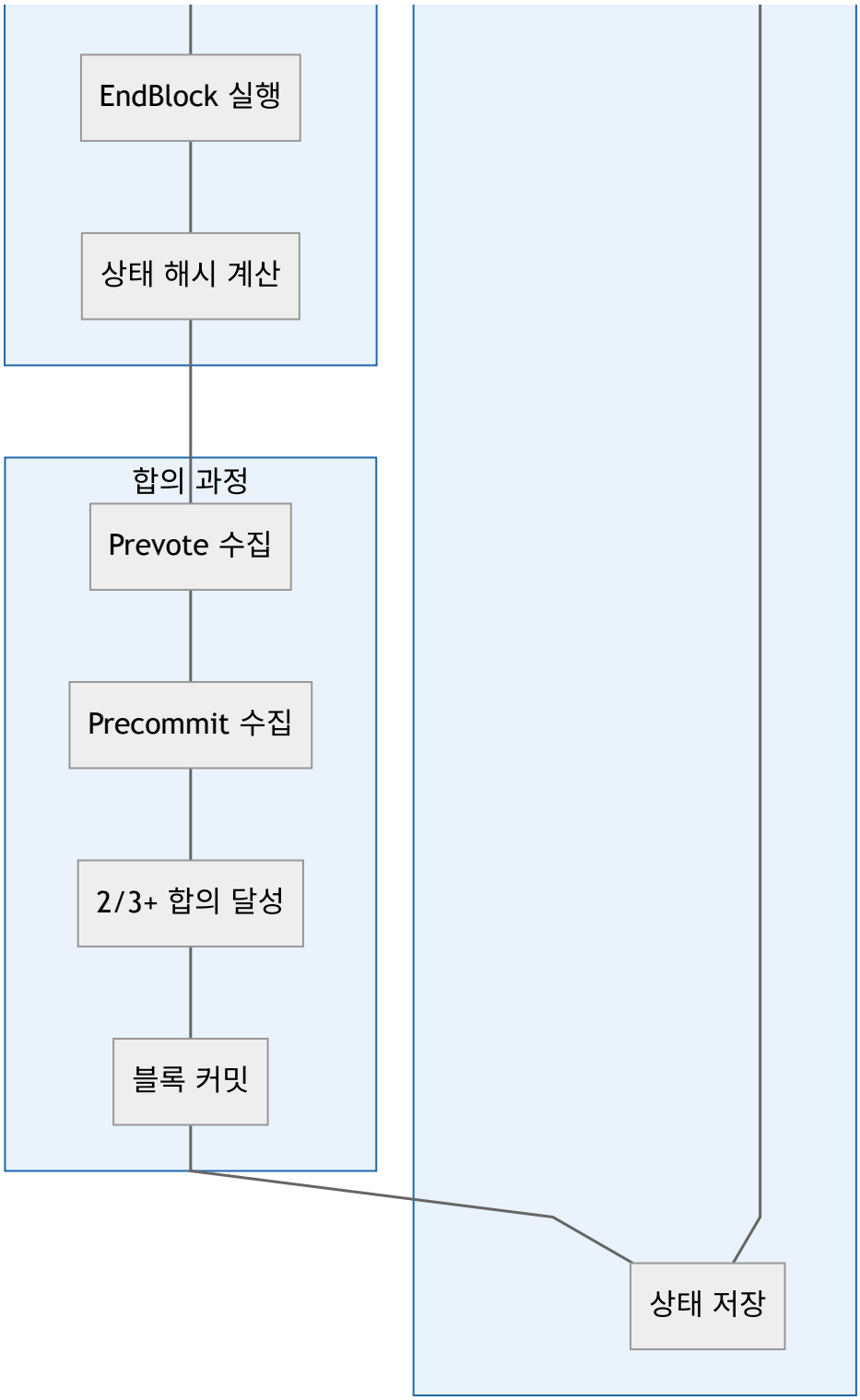
토큰 전송 흐름:

1. **송신**: 토큰 에스크로 → 패킷 생성 → 대상 체인 전송
2. **수신**: 패킷 검증 → IBC 토큰 발행 → 수신자 전송
3. **확인**: Acknowledgment 처리 → 에스크로 해제 또는 유지

6. 블록 생성 및 합의 흐름

6.1 블록 라이프사이클


상태 업데이트
이벤트 발행
인덱싱
다음 블록 준비
블록 제안
검증자 선택
트랜잭션 선택
블록 구성
블록 제안
블록 처리
PreBlock 실행
BeginBlock 실행
트랜잭션 실행



6.2 모듈 실행 순서

코드 참조: `app/app_config.go` (라인 150-200)

```
// BeginBlockers 실행 순서 - 블록 시작 시 실행되는 모듈들
BeginBlockers: []string{
minttypes.ModuleName, // 1. 토큰 발행 (인플레이션)
```

```
distrtypes.ModuleName, // 2. 보상 분배
slashingtypes.ModuleName, // 3. 검증자 슬래싱
evidencetypes.ModuleName, // 4. 증거 처리
stakingtypes.ModuleName, // 5. 스테이킹 업데이트
authz.ModuleName, // 6. 권한 부여
epochstypes.ModuleName, // 7. 에포크 관리
ibcexported.ModuleName, // 8. IBC 처리
marsmoduletypes.ModuleName, // 9. Mars 커스텀 로직
},

// EndBlockers 실행 순서 - 블록 종료 시 실행되는 모듈들
EndBlockers: []string{
```

```
govtypes.ModuleName, // 1. 거버넌스 제안 처리
stakingtypes.ModuleName, // 2. 검증자 세트 업데이트
feegrant.ModuleName, // 3. 수수료 부여 정리
group.ModuleName, // 4. 그룹 관련 정리
marsmoduletypes.ModuleName, // 5. Mars 커스텀 정리
},

// InitGenesis 실행 순서 - 체인 시작 시 초기화 순서
InitGenesis: []string{
consensustypes.ModuleName, // 1. 합의 파라미터 (최우선)
authtypes.ModuleName, // 2. 계정 시스템
banktypes.ModuleName, // 3. 토큰 시스템
distrtypes.ModuleName, // 4. 보상 분배 시스템
stakingtypes.ModuleName, // 5. 스테이킹 시스템
slashingtypes.ModuleName, // 6. 슬래싱 시스템
govtypes.ModuleName, // 7. 거버넌스 시스템
minttypes.ModuleName, // 8. 토큰 발행 시스템
genutiltypes.ModuleName, // 9. 제네시스 트랜잭션 (스테이킹 후)
// ... 기타 모듈들
marsmoduletypes.ModuleName, // 마지막: Mars 커스텀 모듈
}
```

실행 순서의 중요성:

1. **BeginBlockers 순서 규칙**:

- **Mint → Distribution**: 새로운 토큰 발행 후 분배
- **Distribution → Slashing**: 분배 완료 후 슬래싱으로 `CanWithdrawInvariant` 유지
- **Slashing → Staking**: 처벌 적용 후 검증자 상태 업데이트
- **IBC 모듈**: 크로스체인 패킷 처리
- **커스텀 모듈**: 마지막에 실행하여 모든 시스템 상태 반영

2. **EndBlockers 순서 규칙**:

- **Governance**: 제안 처리 및 파라미터 변경 적용
- **Staking**: 검증자 세트 변경사항 적용 (다음 블록에 반영)
- **정리 모듈들**: 임시 상태 정리 및 다음 블록 준비

3. **InitGenesis 순서 규칙**:

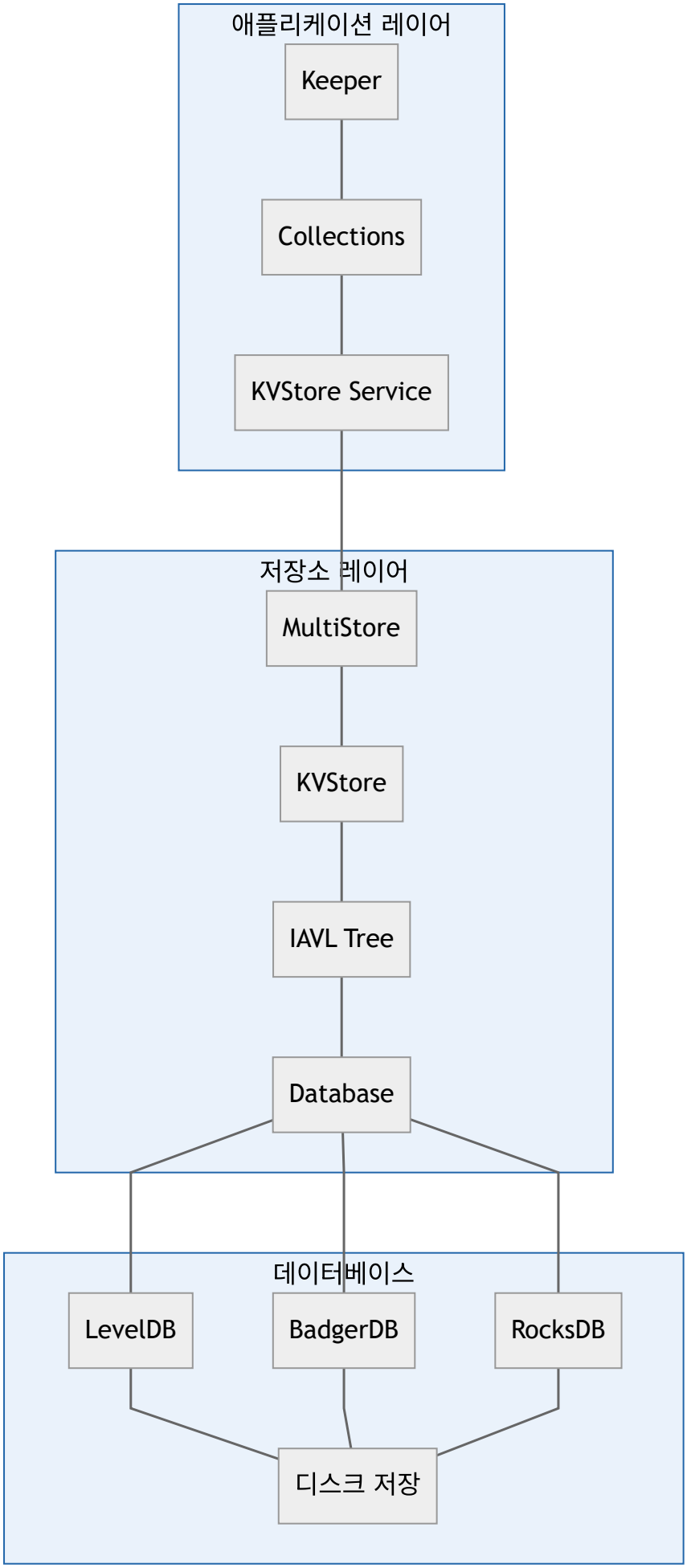
- **Consensus**: 합의 파라미터 우선 설정
- **Auth → Bank**: 계정 시스템 후 토큰 시스템
- **Staking → GenUtil**: 스테이킹 풀 초기화 후 제네시스 트랜잭션 처리
- **의존성 순서**: 의존하는 모듈이 먼저 초기화

불변성 유지:

- **CanWithdrawInvariant**: 분배 모듈에서 인출 가능한 토큰 양이 실제 보유량과 일치
- **BondedPoolInvariant**: 본드된 풀의 토큰 양이 활성 검증자들의 총 스테이킹 양과 일치
- **SupplyInvariant**: 총 공급량이 모든 계정의 잔액 합과 일치

7. 상태 관리 및 저장소 구조

7.1 저장소 계층 구조


애플리케이션 레이어
Keeper
Collections
KVStore Service
저장소 레이어
MultiStore
KVStore
IAVL Tree
Database
데이터베이스
LevelDB
BadgerDB
RocksDB
디스크 저장

7.2 Mars 모듈 상태 구조

코드 참조: `x/mars/keeper/keeper.go` (라인 25-45)

```
// Collections 기반 상태 저장소
type Keeper struct {
// 저장소 서비스 - 실제 KV 저장소에 대한 접근
storeService corestore.KVStoreService
// 코덱 - 데이터 직렬화/역직렬화
cdc codec.Codec
// 주소 코덱 - 바이트 주소와 문자열 주소 간 변환
addressCodec address.Codec
// 권한 주소 - 파라미터 업데이트 권한을 가진 주소 (일반적으로 gov 모듈)
authority []byte
// Collections 스키마 - 타입 안전한 저장소 정의
Schema collections.Schema
// 파라미터 저장소 - 모듈 설정값들을 저장
Params collections.Item[types.Params]
}

// NewKeeper에서 파라미터 저장소 초기화
Params: collections.NewItem(
sb, // 스키마 빌더
types.ParamsKey, // 저장소 키 ("p_mars")
```

```
"params", // 네임스페이스
codec.CollValue[types.Params](cdc) // 값 코덱
)
```

저장소 키 구조:

- **Module Prefix**: `"mars"` (모듈 저장소 구분)
- **Params Key**: `"p_mars"` (파라미터 전용 키)
- **Namespace**: `"params"` (컬렉션 이름)
- **Value Type**: `types.Params` (파라미터 구조체)

Collections의 장점:

1. **타입 안전성**: 컴파일 타임에 타입 검증
2. **자동 직렬화**: 코덱을 통한 자동 인코딩/디코딩
3. **키 관리**: 자동 키 생성 및 충돌 방지
4. **성능 최적화**: 효율적인 저장소 접근 패턴

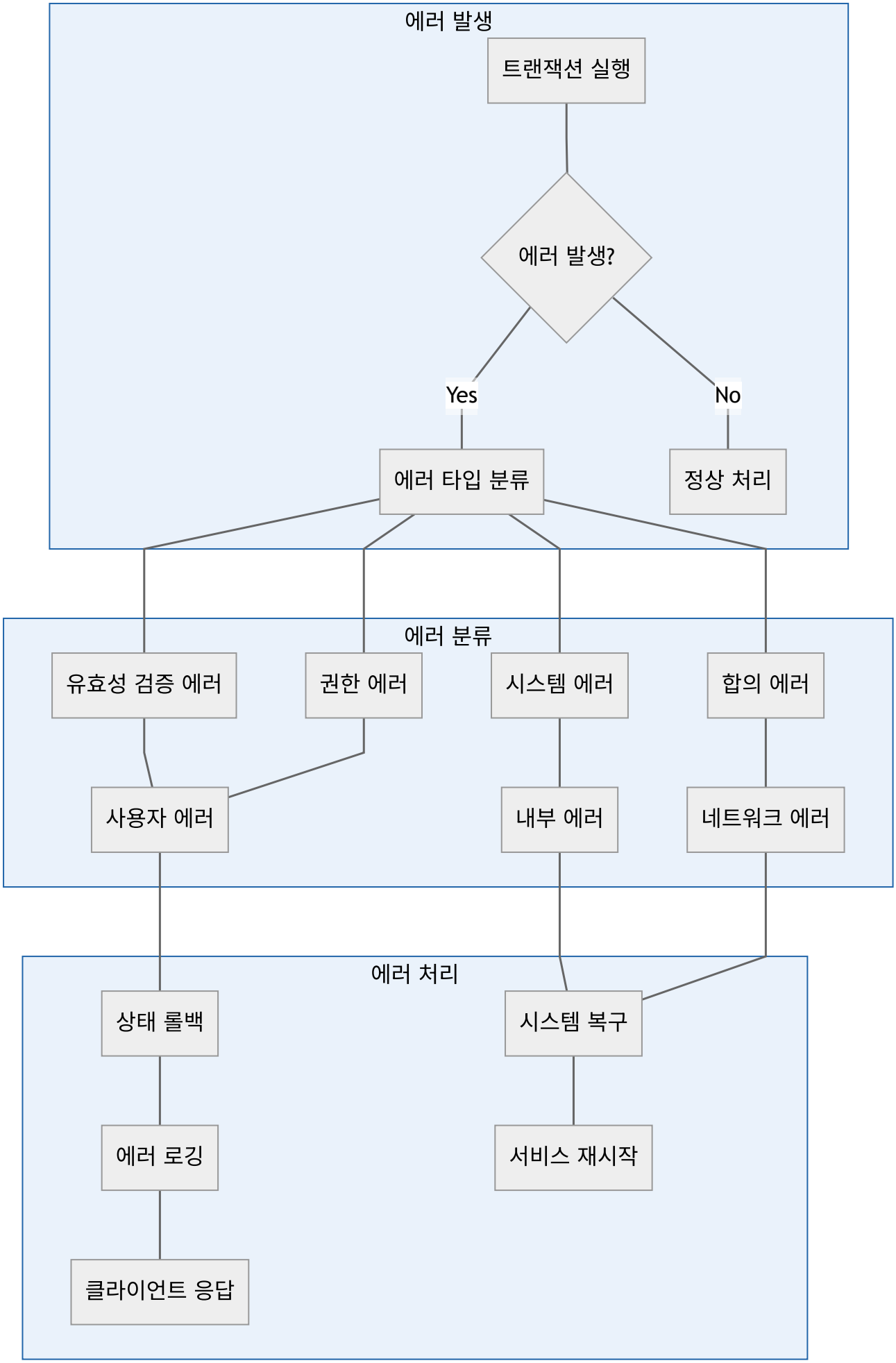
상태 접근 패턴:

```
// 파라미터 조회
params, err := k.Params.Get(ctx)

// 파라미터 저장
err := k.Params.Set(ctx, newParams)

// 파라미터 존재 여부 확인
```

```
exists := k.Params.Has(ctx)
```

8. 에러 처리 및 복구 메커니즘

8.1 에러 처리 흐름


에러 발생
트랜잭션 실행
에러 발생?
Yes
No
에러 타입 분류
정상 처리
에러 분류
유효성 검증 에러
권한 에러
시스템 에러
합의 에러
사용자 에러
내부 에러
네트워크 에러
에러 처리
상태 롤백
시스템 복구
에러 로깅
서비스 재시작
클라이언트 응답

8.2 복구 메커니즘

패닉 복구:

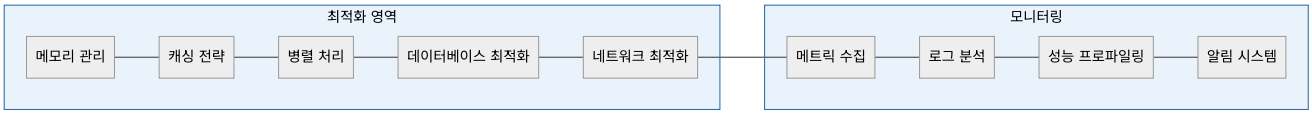
```
// 초기화 단계에서의 패닉 처리
if err := depinject.Inject(appConfig, ...); err != nil {
panic(err) // 복구 불가능한 상황
}
```

트랜잭션 롤백:

- 트랜잭션 실행 중 에러 발생 시 상태 자동 롤백
- 가스 소모는 유지하되 상태 변경은 취소

9. 성능 최적화 및 모니터링

9.1 성능 최적화 포인트



9.2 낙관적 실행 (Optimistic Execution)

코드 참조: `app/app.go`

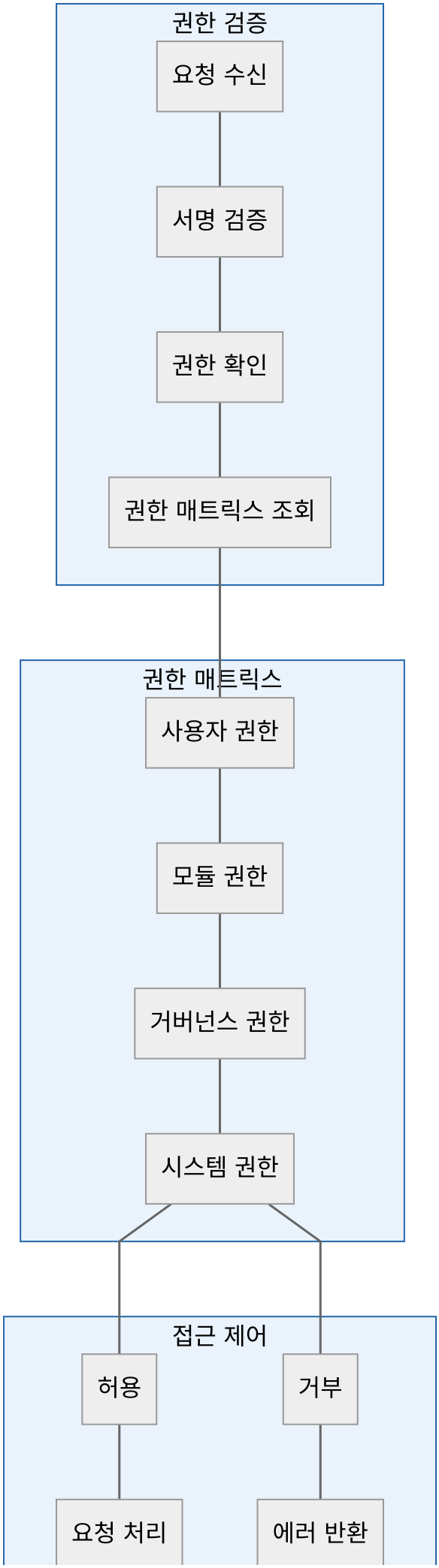
```
// 낙관적 실행 활성화
baseAppOptions = append(baseAppOptions, baseapp.SetOptimisticExecution())
```

장점:

- 트랜잭션 병렬 실행으로 성능 향상
- 의존성이 없는 트랜잭션들의 동시 처리
- 전체 블록 처리 시간 단축

10. 보안 및 권한 관리

10.1 권한 관리 흐름


권한 검증
요청 수신
서명 검증
권한 확인
권한 매트릭스 조회
권한 매트릭스
사용자 권한
모듈 권한
거버넌스 권한
시스템 권한
접근 제어
허용
거부
요청 처리
에러 반환

10.2 모듈 계정 권한

코드 참조: `app/app_config.go`

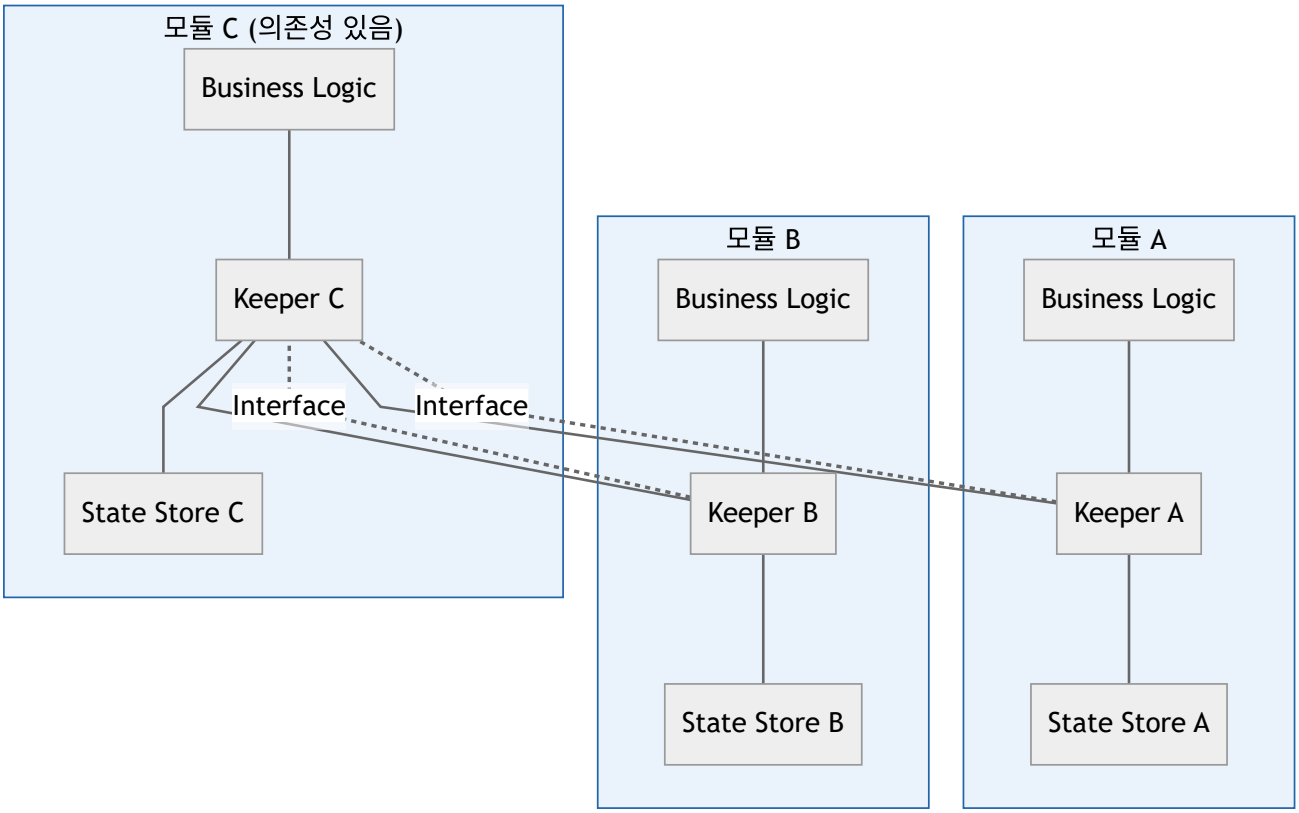
```
// 모듈 계정 권한 설정
moduleAccPerms = []*authmodulev1.ModuleAccountPermission{
{Account: minttypes.ModuleName, Permissions: []string{authtypes.Minter}},
{Account: stakingtypes.BondedPoolName, Permissions: []string{authtypes.Burner, s
// ... 기타 권한 설정
}
```

권한 타입:

- **Minter**: 토큰 발행 권한
- **Burner**: 토큰 소각 권한
- **Module**: 모듈별 특수 권한

11. 모듈 간 상호작용 패턴

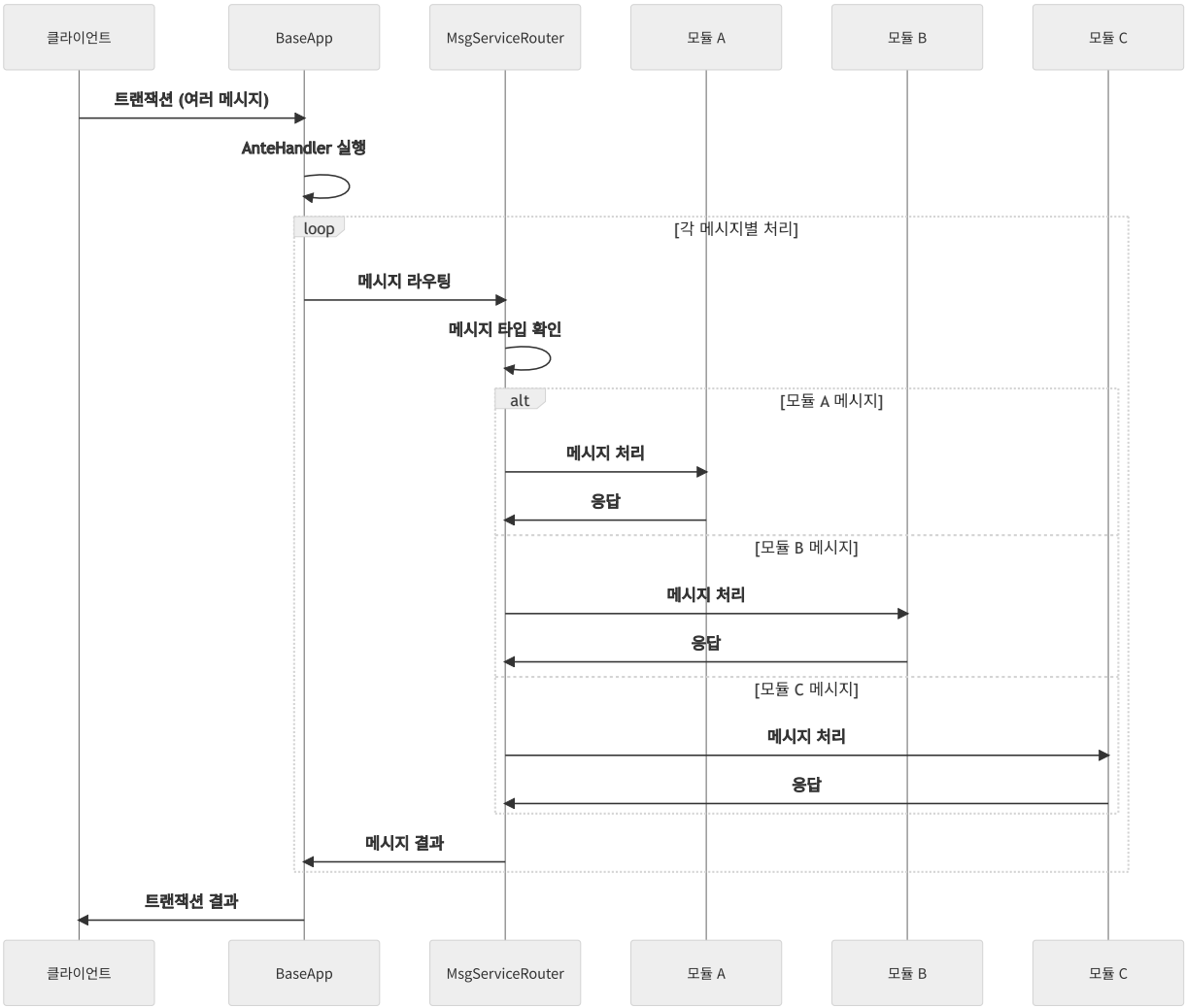
11.1 Keeper 패턴을 통한 모듈 통신



Keeper 의존성 주입 예시 (`app/app.go`):

```
// 의존성 주입 시 Keeper 간 참조 설정
if err := depinject.Inject(appConfig,
&app.AuthKeeper, // 기본 모듈
&app.BankKeeper, // AuthKeeper에 의존
&app.StakingKeeper, // BankKeeper에 의존
&app.GovKeeper, // 모든 모듈에 의존 (거버넌스)
&app.MarsKeeper, // 필요한 모듈들에 의존
); err != nil {
panic(err)
}
```

11.2 메시지 라우팅 패턴

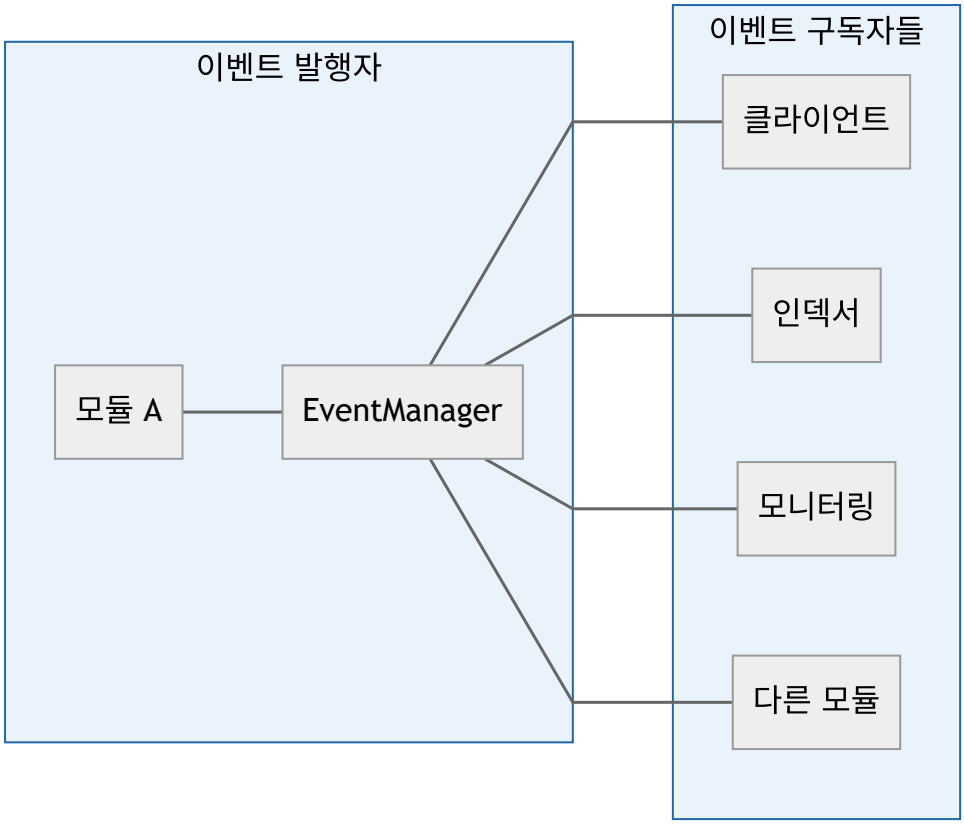

클라이언트
BaseApp
MsgServiceRouter
모듈 A
모듈 B
모듈 C
트랜잭션 (여러 메시지)
AnteHandler 실행
loop
[각 메시지별 처리]
메시지 라우팅
메시지 타입 확인
alt
[모듈 A 메시지]
메시지 처리
응답
[모듈 B 메시지]
메시지 처리
응답
[모듈 C 메시지]
메시지 처리
응답
메시지 결과
트랜잭션 결과
클라이언트
BaseApp
MsgServiceRouter
모듈 A
모듈 B
모듈 C

11.3 이벤트 기반 통신 패턴

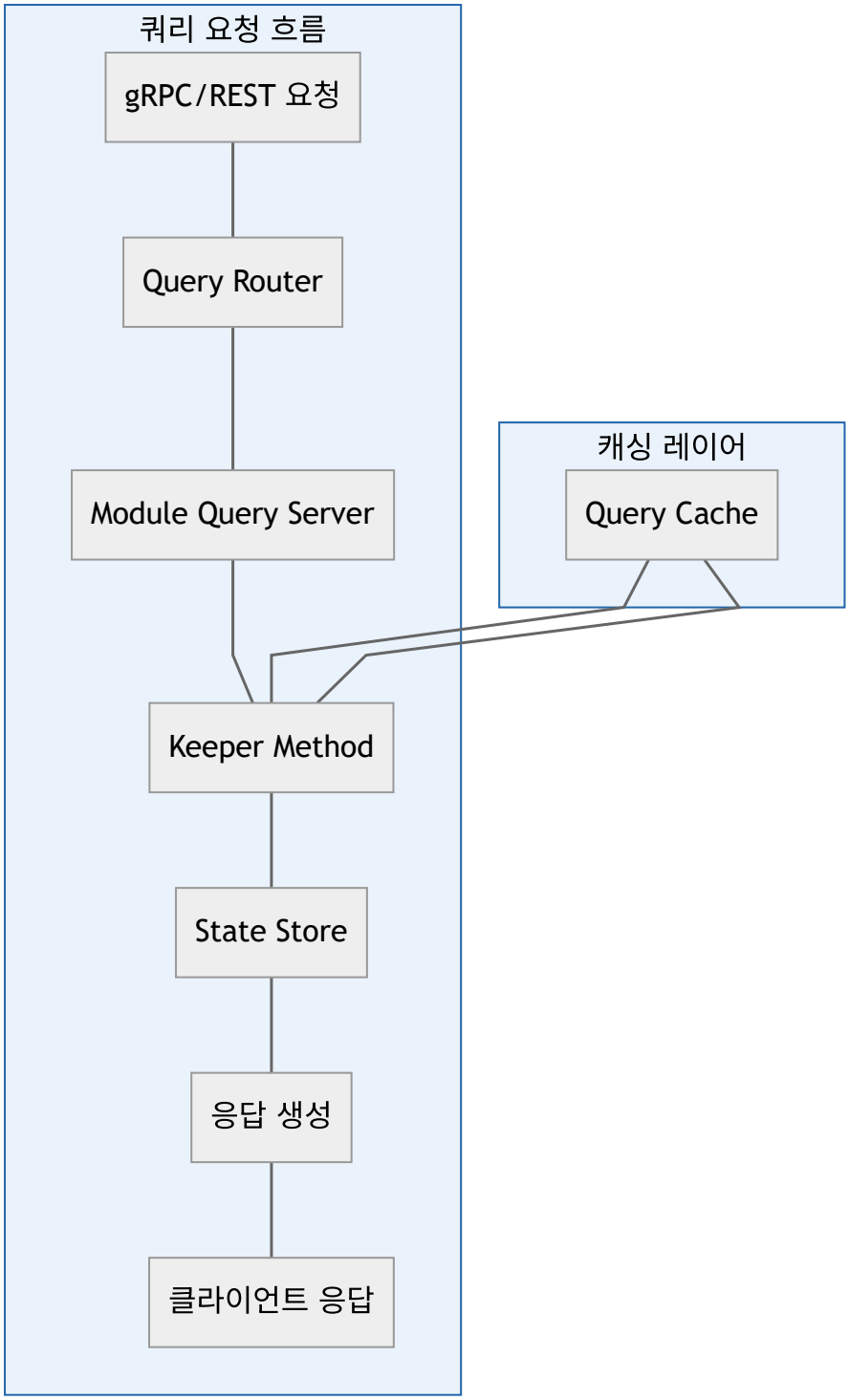
이벤트 발행 (`x/mars/keeper/msg_update_params.go`):

```go
func (k msgServer) UpdateParams(ctx context.Context, req *types.MsgUpdateParams)
// 파라미터 업데이트 로직
if err := k.Params.Set(ctx, req.Params); err != nil {
return  nil, err
}
// 이벤트 발행
sdkCtx := sdk.UnwrapSDKContext(ctx)
sdkCtx.EventManager().EmitEvent(
sdk.NewEvent(
types.EventTypeParamsUpdated,
sdk.NewAttribute(types.AttributeKeyUpdatedBy, req.Authority),
sdk.NewAttribute(types.AttributeKeyNewParams, req.Params.String()),
),
)
return &types.MsgUpdateParamsResponse{}, nil
}
```

이벤트 구독 패턴:



11.4 쿼리 처리 패턴



쿼리 서버 구현 (`x/mars/keeper/query.go`):

```
func (k Keeper) Params(ctx context.Context, req *types.QueryParamsRequest) (*typ

if req == nil {

return  nil, status.Error(codes.InvalidArgument, "invalid request")

}
```

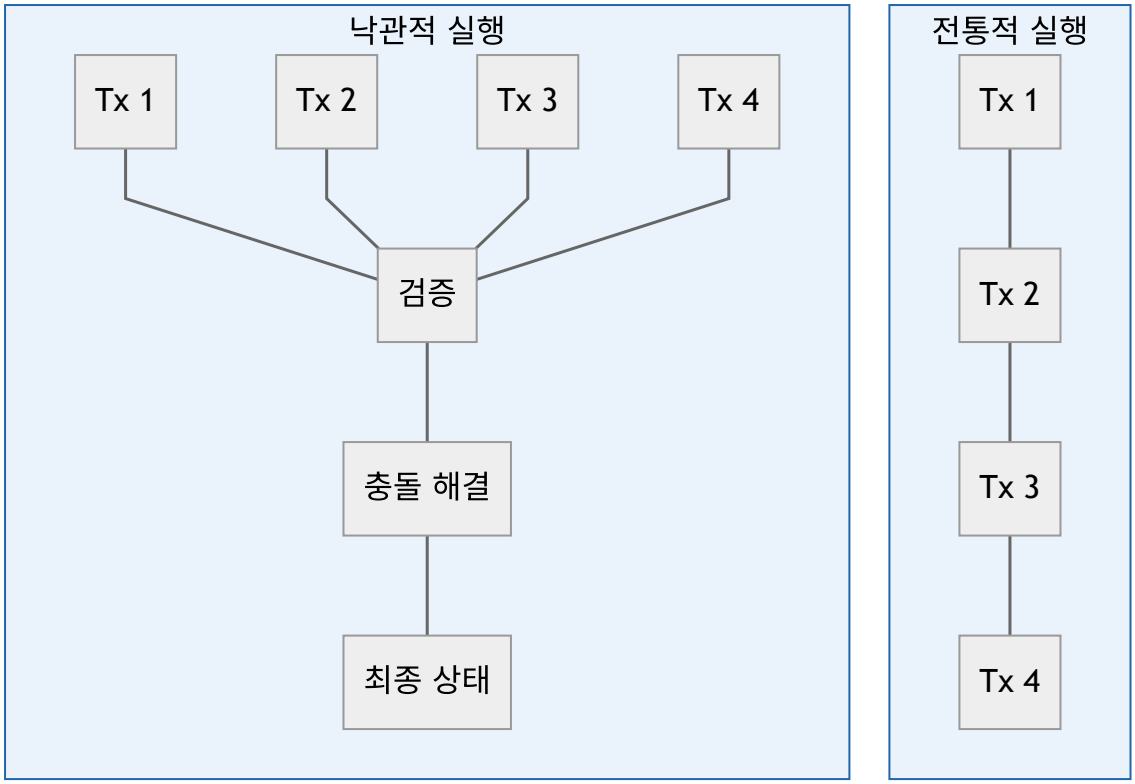
```go
// 상태에서 파라미터 조회
params, err := k.Params.Get(ctx)
if err != nil {
if errors.Is(err, collections.ErrNotFound) {
return &types.QueryParamsResponse{Params: types.DefaultParams()}, nil
}
return  nil, status.Error(codes.Internal, err.Error())
}
return &types.QueryParamsResponse{Params: params}, nil
}
```

12. 성능 최적화 및 확장성

12.1 낙관적 실행 (Optimistic Execution)

코드 참조: `app/app.go` (라인 280)

```go
// 낙관적 실행 활성화
baseAppOptions = append(baseAppOptions, baseapp.SetOptimisticExecution())
```



장점:

- 독립적인 트랜잭션들의 병렬 실행
- 전체 블록 처리 시간 단축
- 높은 처리량 달성

12.2 Collections 기반 상태 최적화

타입 안전한 저장소 접근:

```
// 기존 방식 (타입 불안전)
store := ctx.KVStore(k.storeKey)
bz := store.Get([]byte("params"))
var params types.Params
k.cdc.MustUnmarshal(bz, &params)
```

```
// Collections 방식 (타입 안전)
params, err := k.Params.Get(ctx)
```

성능 이점:

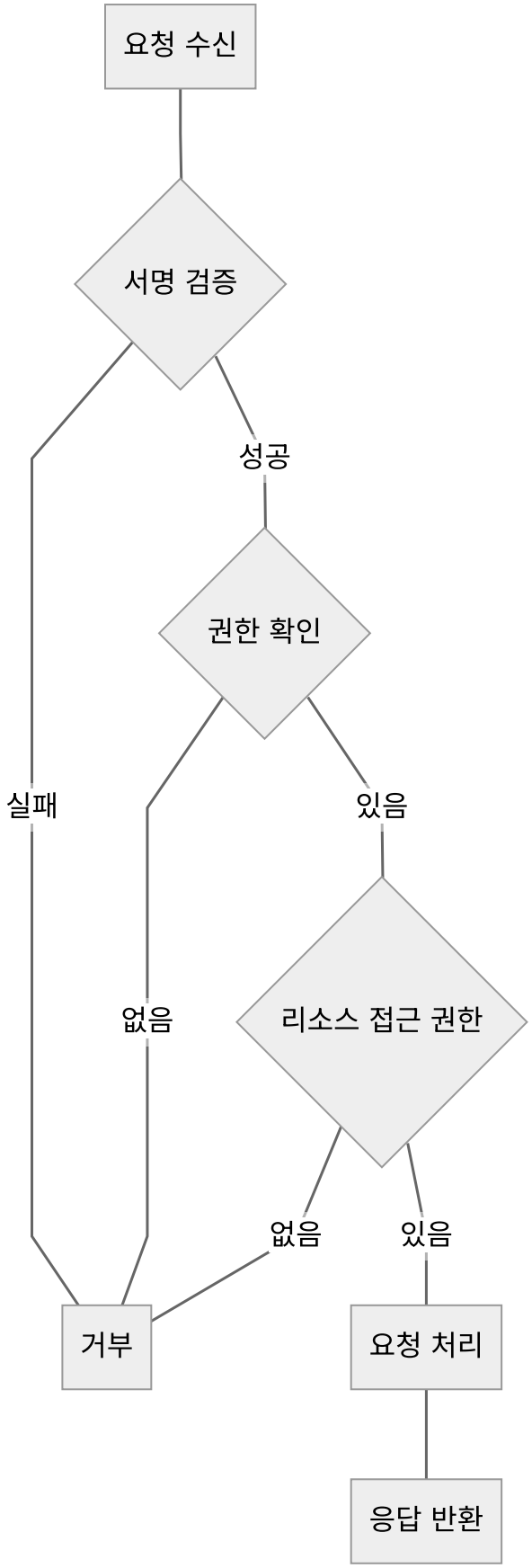
- 컴파일 타임 타입 검증
- 자동 직렬화/역직렬화
- 키 충돌 방지
- 효율적인 메모리 사용

13. 보안 및 권한 관리 심화

13.1 모듈 권한 매트릭스

모듈	권한 타입	권한 주체	설명
Auth	계정 생성/관리	시스템	자동 계정 생성 및 시퀀스 관리
Bank	토큰 전송	사용자/모듈	잔액 확인 후 전송 허용
Staking	위임/언본딩	사용자	최소 위임량 및 언본딩 기간 검증
Gov	제안/투표	사용자/검증자	보증금 및 투표권 검증
Mars	파라미터 업데이트	Gov 모듈	Authority 주소 검증

13.2 권한 검증 흐름



결론

Mars 블록체인 애플리케이션은 Cosmos SDK의 모범 사례를 따라 구현된 현대적인 블록체인 시스템입니다. 주요 특징은 다음과 같습니다:

핵심 아키텍처 특징

1. **의존성 주입 기반 모듈화**: `depinject` 시스템을 통한 자동 의존성 해결
2. **Collections 기반 상태 관리**: 타입 안전한 저장소 시스템
3. **IBC 크로스체인 지원**: Inter-Blockchain Communication을 통한 상호 운용성
4. **낙관적 실행**: 트랜잭션 병렬 처리를 통한 성능 최적화

전체 흐름 요약

```
초기화 → 서버 시작 → 요청 처리 → 블록 생성 → 상태 업데이트
↑                                                ↓
←←←←←←←←←←← 순환 구조 ←←←←←←←←←←←←←←←←←←←←←←←←
```

각 단계에서 적절한 에러 처리와 보안 검증을 통해 시스템의 무결성을 보장하며, 모듈 간 명확한 책임 분리를 통해 유지보수성과 확장성을 확보했습니다.

개발자를 위한 핵심 포인트

- **모듈 개발**: Keeper 패턴과 Collections 사용
- **상태 관리**: 타입 안전한 저장소 접근
- **메시지 처리**: 권한 검증과 상태 변경 분리
- **쿼리 구현**: 효율적인 데이터 조회 패턴
- **테스트**: 시뮬레이션과 단위 테스트 활용